

BÀI THỰC HÀNH

MINI LAB

DATABASE TESTING CƠ BẢN & TÍCH HỢP AI/MCP

Môn học

Kiểm thử Phần mềm / Software Quality Assurance

Chủ đề

Kiểm thử Cơ sở dữ liệu Hệ thống E-Commerce · Phiên bản rút gọn

Technical

Node.js · PostgreSQL · Jest · Supertest

Công cụ AI

MCP Database Server · Schema Analysis · EXPLAIN ANALYZE

Sản phẩm

db-tests.test.js · performance.sql · REPORT.md

Mục lục

1 Thông tin chung	1
1.1 Technical và sản phẩm nộp	1
2 Cấu trúc cơ sở dữ liệu và đối tượng kiểm thử	1
2.1 Các đối tượng chứa lỗi logic cần phát hiện	2
3 Quy trình thực hiện	2
3.1 Bước 1 — Phân tích schema qua MCP Database Server	2
3.2 Bước 2 — Khởi tạo dữ liệu kiểm thử tối thiểu	3
3.3 Bước 3 — Kiểm thử tầng Database	3
3.4 Bước 4 — Kiểm thử API và bảo mật	4
3.5 Bước 5 — Phân tích hiệu năng với EXPLAIN ANALYZE	4
4 Bảng tổng hợp 7 khía cạnh kiểm thử	5
5 Cấu trúc báo cáo REPORT.md	5
6 Tiêu chí đánh giá và trọng số	6
7 Checklist trước khi nộp	6

1. Thông tin chung

Mini lab tập trung vào kiểm thử cơ sở dữ liệu PostgreSQL cho hệ thống E-Commerce thu gọn. Sinh viên sẽ kiểm thử trực tiếp tầng database, kiểm thử tích hợp API, phân tích hiệu năng và sử dụng AI Agent thông qua MCP Database Server để hỗ trợ khám phá schema.

Mục tiêu bài thực hành

Kiểm tra constraints, function, trigger, stored procedure, transaction, API nghiệp vụ, SQL Injection, quyền truy cập và kế hoạch thực thi truy vấn; đồng thời ghi nhận kết quả AI/MCP thành bằng chứng có thể truy vết và kiểm chứng.

1.1. Technical và sản phẩm nộp

Thành phần / Tập	Vai trò / Nội dung
Node.js, Jest, Supertest	Chạy test suite và kiểm thử tích hợp API.
PostgreSQL	Chứa schema và các đối tượng database cần kiểm thử.
MCP Database Server	Phân tích schema và hỗ trợ đọc kế hoạch truy vấn.
db-tests.test.js	Test tự động tầng database, API và security.
performance.sql	Truy vấn EXPLAIN ANALYZE, tạo index và so sánh.
REPORT.md	Báo cáo lỗi, hiệu năng và nhật ký thao tác MCP.

Sử dụng `@modelcontextprotocol/server-postgres` để hỗ trợ liệt kê bảng, khóa ngoại, constraints, trigger, function, stored procedure và đọc EXPLAIN ANALYZE. Phản hồi AI chỉ là dữ liệu hỗ trợ; sinh viên phải kiểm chứng bằng SQL hoặc test tự động.

Phạm vi

Bài lab bao phủ 7 khía cạnh: **Schema, Functional, Trigger, Stored Procedure, Function, Performance và Security**

2. Cấu trúc cơ sở dữ liệu và đối tượng kiểm thử

Schema PostgreSQL ban đầu

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  role VARCHAR(20) DEFAULT 'customer'  
);  
  
CREATE TABLE products (  
  id SERIAL PRIMARY KEY,
```

```

name VARCHAR(255) NOT NULL,
price NUMERIC(10, 2) NOT NULL,
stock INT DEFAULT 0 CHECK (stock >= 0)
);

CREATE TABLE coupons (
  id SERIAL PRIMARY KEY,
  code VARCHAR(50) UNIQUE NOT NULL,
  discount_type VARCHAR(20)
    CHECK (discount_type IN ('percent', 'fixed')) NOT NULL,
  discount_value NUMERIC(10, 2) NOT NULL,
  expired_at TIMESTAMP NOT NULL,
  is_active INT DEFAULT 1
);

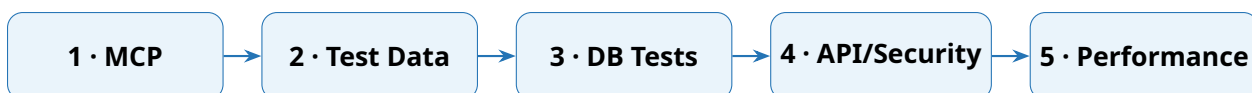
CREATE TABLE orders (
  id SERIAL PRIMARY KEY,
  user_id INT REFERENCES users(id) ON DELETE CASCADE,
  total_amount NUMERIC(10, 2) NOT NULL,
  final_amount NUMERIC(10, 2) NOT NULL,
  status VARCHAR(20)
    CHECK (status IN
      ('pending', 'confirmed', 'shipping', 'delivered', 'canceled'))
    DEFAULT 'pending'
);

```

2.1. Các đối tượng chứa lỗi logic cần phát hiện

1. `fn_calculate_discount(type, value, order_amount)`: hàm tính mức giảm giá có thể vượt tổng đơn hàng.
2. `sp_process_checkout(...)`: thủ tục có nguy cơ không rollback toàn bộ khi hết hàng.
3. `trg_prevent_negative_stock`: cần xác minh thực sự chặn cập nhật tồn kho âm.

3. Quy trình thực hiện



3.1. Bước 1 — Phân tích schema qua MCP Database Server

1. Kết nối AI Agent với PostgreSQL qua MCP Database Server.
2. Trích xuất bảng, khóa ngoại, constraints, trigger, function và stored procedure.
3. Ghi prompt và tóm tắt phản hồi vào `REPORT.md`.

“Liệt kê danh sách bảng, khóa ngoại, ràng buộc, trigger, function và stored procedure trong schema hiện tại. Với mỗi đối tượng, nêu tên, bảng liên quan và mục đích chính.”

3.2. Bước 2 — Khởi tạo dữ liệu kiểm thử tối thiểu

Bảng	Số lượng	Yêu cầu
users	5	Email duy nhất; role phù hợp cho test RBAC.
products	5	Có sản phẩm còn hàng và ít nhất một sản phẩm hết hàng.
orders	Khoảng 200	Liên kết ngẫu nhiên với user_id.
coupons	4	CP_OK, CP_EXPIRED, CP_INACTIVE, CP_PERCENT150.

3.3. Bước 3 — Kiểm thử tầng Database

Tập: `db-tests.test.js`

Function, Trigger, Stored Procedure và Constraint

```
// 1. Function Testing
it('không cho phép giá trị giảm giá vượt quá 100%', async () => {
  const r = await db.query(
    `SELECT fn_calculate_discount('percent', 150, 200) AS d`
  );
  expect(Number(r.rows[0].d)).toBeLessThanOrEqual(200);
});

// 2. Trigger Testing
it('chặn cập nhật làm stock âm', async () => {
  await expect(
    db.query(`UPDATE products SET stock = -5 WHERE id = 1`)
  ).rejects.toThrow();
});

// 3. Stored Procedure Testing (Atomicity)
it('rollback khi có sản phẩm hết hàng trong đơn', async () => {
  const before = await db.query(`SELECT stock FROM products WHERE id = 1`);
  await expect(
    sp_process_checkout(userId, [1, 2, outOfStockId])
  ).rejects.toThrow();
  const after = await db.query(`SELECT stock FROM products WHERE id = 1`);
  expect(after.rows[0].stock).toBe(before.rows[0].stock);
});

// 4. Schema / Constraint Testing
it('từ chối email trùng UNIQUE', async () => {
  await db.query(`INSERT INTO users(email) VALUES ('a@x.com')`);
  await expect(
    db.query(`INSERT INTO users(email) VALUES ('a@x.com')`)
  ).rejects.toThrow();
});
```

Atomicity cần chứng minh

Sau lỗi, tồn kho của mọi sản phẩm đã xử lý phải trở về giá trị ban đầu và không có đơn hàng dở dang. Đây là kiểm thử tính **nguyên tử** của transaction.

3.4. Bước 4 — Kiểm thử API và bảo mật

Functional API, SQL Injection và RBAC

```
it('từ chối coupon hết hạn', async () => {
  const res = await request(app).post('/api/apply-coupon')
    .send({ code: 'CP_EXPIRED', order_amount: 300 });
  expect(res.status).toBe(400);
});

it('chặn canceled chuyển sang delivered', async () => {
  const res = await request(app)
    .put(`/api/admin/orders/${canceledOrderId}/status`)
    .send({ status: 'delivered' });
  expect(res.status).toBe(400);
});

it('API tìm kiếm chống SQL Injection', async () => {
  const res = await request(app)
    .get(`/api/products/search?q=' OR '1'='1`);
  expect(res.status).not.toBe(500);
  const count = await db.query(`SELECT COUNT(*) FROM products`);
  expect(Number(count.rows[0].count)).toBe(5);
});

it('app_user không có quyền DROP TABLE', async () => {
  await expect(
    appUserDb.query(`DROP TABLE products`)
  ).rejects.toThrow(/permission denied/i);
});
```

Lưu ý

HTTP 500 không phải tiêu chí duy nhất cho SQL Injection. Cần kiểm tra parameterized query, phạm vi kết quả trả về và xác minh dữ liệu không bị sửa hoặc xóa.

3.5. Bước 5 — Phân tích hiệu năng với EXPLAIN ANALYZE

Tệp: performance.sql

Đo trước và sau khi tạo index

```
-- 1. Đo trước khi tạo index
EXPLAIN (ANALYZE, BUFFERS)
SELECT user_id, SUM(final_amount)
FROM orders
GROUP BY user_id
ORDER BY SUM(final_amount) DESC;

-- 2. Tạo index và cập nhật thống kê
CREATE INDEX idx_orders_user_id ON orders(user_id);
ANALYZE orders;
```

```
-- 3. Đo lại sau khi tạo index
EXPLAIN (ANALYZE, BUFFERS)
SELECT user_id, SUM(final_amount)
FROM orders
GROUP BY user_id
ORDER BY SUM(final_amount) DESC;
```

Diễn giải kết quả

Với khoảng 200 bản ghi và truy vấn tổng hợp toàn bảng, PostgreSQL có thể vẫn chọn Seq Scan sau khi tạo index. Đây không nhất thiết là lỗi. Báo cáo phải ghi kế hoạch thực tế, Execution Time, buffers và giải thích lựa chọn của planner.

4. Bảng tổng hợp 7 khía cạnh kiểm thử

Khía cạnh	Kịch bản rút gọn	Công cụ
Schema	Vi phạm UNIQUE trên email.	Jest + DB Client
Functional	Coupon hết hạn; chuyển trạng thái sai.	Jest + Supertest
Trigger	Cập nhật tồn kho thành số âm.	Jest + DB Client
Stored Procedure	Tính nguyên tử và rollback khi hết hàng.	Jest + DB Client
Function	Chiết khấu vượt quá 100%.	SQL / Jest
Security	SQL Injection và RBAC.	Jest + Supertest
Performance	So sánh kế hoạch trước/sau index.	SQL + MCP Server

5. Cấu trúc báo cáo REPORT.md

Mẫu báo cáo

```
# BÁO CÁO KẾT QUẢ KIỂM THỬ

## 1. Tổng quan Test Run
- Môi trường:
- Tổng số test / Pass / Fail:

## 2. Danh sách Lỗi Phát hiện
1. fn_calculate_discount
  - Expected / Actual / Root cause / Fix đề xuất
2. sp_process_checkout
  - Dữ liệu trước và sau transaction
3. trg_prevent_negative_stock: [Đạt / Không đạt]

## 3. Kết quả Hiệu năng
- Trước index: X ms; Scan: ...; Buffers: ...
- Sau index: Y ms; Scan: ...; Buffers: ...
- Nhận xét về PostgreSQL planner:
```

4. Nhật ký MCP

- Prompt: "Liệt kê bảng, khóa ngoại, trigger, function, stored procedure."
- Tóm tắt phản hồi:
- Cách kiểm chứng:

5. Kết luận và Khuyến nghị**Yêu cầu bằng chứng**

Mỗi lỗi cần có input, expected result, actual result, test case và root cause. Báo cáo phải phân biệt nội dung AI đề xuất với kết luận đã được sinh viên kiểm chứng.

6. Tiêu chí đánh giá và trọng số

Tiêu chí	Trọng số
DB Unit Testing: Function / Trigger / Stored Procedure / Schema	40%
Functional & Security Testing: API / SQLi / RBAC	25%
Performance Testing: EXPLAIN ANALYZE và index	15%
Tích hợp MCP và nhật ký phân tích	10%
Trình bày REPORT.md	10%
Tổng cộng	100%

7. Checklist trước khi nộp

- db-tests.test.js chạy được và có assertion rõ ràng.
- performance.sql có phép đo trước/sau index.
- REPORT.md có Pass/Fail, bugs, hiệu năng và MCP Log.
- Test thay đổi dữ liệu có setup/cleanup hoặc rollback.
- Không đưa password hoặc connection string nhạy cảm vào bài nộp.
- Kết luận AI/MCP đã được đối chiếu với kết quả thực thi.

Cấu trúc thư mục nộp bài

```
STUDENT_ID_MINILAB_POSTGRES/
├─ db-tests.test.js
├─ performance.sql
└─ REPORT.md
```

— Hết đề —

Dùng test để tạo artifacts; dùng AI để hỗ trợ phân tích; dùng dữ liệu để kết luận.